LATIHAN TERSTRUKTUR (PERTEMUAN 4)

IMPLEMENTASI UNIT TESTING DENGAN GITHUB ACTIONS



Nama: Muhammad Irfan Abdul Aziz Ramdani

NIM: 1224014

Kelas: Teknik Informatika — STMIK Bandung

Tanggal: 10 Oktober 2026

Repository: https://github.com/Muhirabazram/latihan-git-1224014

BAGIAN A: GIT & GITHUB DASAR (25 POIN)

Soal 1 (10 Poin)

1. Centralized VCS vs Distributed VCS

Aspek	Centralized VCS (CVCS)	Distributed VCS (DVCS)
Arsitektur	Satu server pusat menyimpan semua versi/history. Client hanya punya working copy (checkout).	Setiap developer punya salinan lengkap repository + history.
Ketergantungan server	Harus online/terhubung ke server untuk commit, log, diff history.	Bisa commit, branch, log, diff secara offline. Sinkronisasi (push/pull) hanya saat bertukar perubahan.
Single point of failure	Ya. Jika server rusak/hilang dan belum ada backup, history berisiko hilang.	Tidak. Setiap clone adalah backup penuh.
Branching/merging	Umumnya berat (server-side, mahal).	Ringan dan cepat (lokal dulu).
Contoh	1. Apache Subversion (SVN) 2. CVS (Concurrent Versions System) / Perforce Helix Core	1. Git 2. Mercurial

2. Lima Git commands wajib + fungsi

1. `git clone <url>` — menyalin remote repository ke lokal (sekaligus mengunduh seluruh history).
2. `git add <file>/ .` — memindahkan perubahan ke staging area sebelum commit.
3. `git commit -m "pesan"` — menyimpan snapshot staged ke history lokal dengan pesan yang menjelaskan perubahan.
4. `git push origin <branch>` — mengirim commit lokal ke remote (GitHub).
5. `git pull origin <branch>` — mengambil + menggabungkan perubahan remote ke lokal (`fetch` + `merge`).

- Cadangan yang juga wajib: `git branch`, `git checkout`/`switch`, `git merge`, `git log --oneline`, `git status`.

3. Fungsi `.gitignore`

File teks di root repo yang mendaftarkan pola file/direktori yang harus **diabaikan Git**

(tidak masuk staging/commit). Fungsi: mencegah file build (`target/`), IDE (`.idea/`, `.vscode/`),

OS (`.DS_Store`, `Thumbs.db`), dependency, file coverage (`*.exec`), dan secrets (`.env`)

ter-commit. Membuat repo bersih, kecil, dan aman. Contoh isi untuk Maven ada di `.gitignore` repo ini.

Soal 2 (15 Poin) — Praktik Git Commands

Urutan command yang dipakai di repository ini (dijalankan persis seperti history Git repo):

```
# 1. Buat repository baru di GitHub (via web):
#    https://github.com/new -> nama: latihan-git-1224014 -> Create repository

# 2. Clone ke lokal
git clone https://github.com/Muhirabazram/latihan-git-1224014.git
cd latihan-git-1224014
```

```
# 3-5. README + commit + push ke main
git branch -M main
git add README.md
git commit -m "docs: add README"
git push -u origin main

# 6-7. Buat branch baru dan pindah ke sana
git branch feature-test
git checkout feature-test
# (alternatif satu langkah: git checkout -b feature-test)

# 8. Buat test.txt dan commit
git add test.txt
git commit -m "test: add test file"

# 9. Push branch feature-test
git push -u origin feature-test

# 10. Merge ke main
git checkout main
git merge feature-test
git push origin main
```

Bukti di repo ini:

- Commit `docs: add README` hanya berisi `README.md`.
- Commit `test: add test file` di branch `feature-test` hanya berisi `test.txt`.
- Branch `feature-test` sudah di-merge ke `main` (lihat `git log --oneline --graph`).

BAGIAN B: UNIT TESTING FRAMEWORK (35 POIN)

Soal 3 (15 Poin)

Kode yang diuji: `src/main/java/com/stmik/PendaftaranValidator.java`

1. Lima test case dari independent path

TC	Path	Input (nama, umur, email, setuju)	Expected
TC1	P1 — nama kosong	`("", 20, "user@mail.com", true)`	`"Nama harus diisi"`
TC2	P2 — umur < 17	`("Budi", 16, "budi@mail.com", true)`	`"Umur minimal 17 tahun"`
TC3	P3/P4 — email tidak valid (null / tanpa `@`)	`("Budi", 20, null, true)` dan `("Budi", 20, "budimail.com", true)`	`"Email tidak valid"`
TC4	P5 — tidak setuju syarat	`("Budi", 20, "budi@mail.com", false)`	`"Harus menyetujui syarat dan ketentuan"`
TC5	Semua valid	`("Budi", 20, "budi@mail.com", true)`	`"Pendaftaran berhasil"`

Catatan: P3 (`email == null`) dan P4 (`!contains("@")`) berada di satu `if` dengan `||`, jadi dipecah jadi 2 baris uji agar kedua kondisi `||` tercakup. Total file test berisi 7 `@Test` (termasuk varian nama `null`).

2. Implementasi JUnit

Lihat file: `src/test/java/com/stmik/PendaftaranValidatorTest.java`

Contoh satu method (pola Arrange-Act-Assert):

```
@Test
public void testPath2_UmurKurangDari17() {
    String result = validator.validasiPendaftaran("Budi", 16, "budi@mail.com", true); // Act
    assertEquals("Umur minimal 17 tahun", result); // Assert
}
```

Soal 4 (20 Poin)

Kode yang diuji: `src/main/java/com/stmik/StudentGrade.java`

1. Cyclomatic Complexity `calculateGrade()`

Keputusan (predicate): `score<0`, `score>100`, `score>=80`, `score>=70`, `score>=60`, `score>=50` = 6.

Rumus: `V(G) = P + 1 = 6 + 1 = 7`.

Jika operator `||` pada validasi disederhanakan sebagai 1 keputusan `if`, maka

`V(G) = 5 + 1 = 6`. Nilai yang umum dipakai di kuliah (menghitung tiap `if/else-if`): **6**;

nilai ketat McCabe (menghitung `||`): **7**.

2. Independent Path `calculateGrade()`

6. Skor invalid → `throw IllegalArgumentException` (mis. `-1` / `101`).
7. `score >= 80` → `"A"` (mis. 85).
8. `score >= 70` → `"B"` (mis. 75).
9. `score >= 60` → `"C"` (mis. 65).
10. `score >= 50` → `"D"` (mis. 55).
11. Selain itu → `"E"` (mis. 30).
12. (Ketat) Pembedaan `score < 0` vs `score > 100` sebagai dua path invalid terpisah.

3. Unit test JUnit

Lihat file: `src/test/java/com/stmik/StudentGradeTest.java`

Mencakup `calculateGrade` (A/B/C/D/E, boundary 80/79, invalid -1/101)

dan `isPassed` (true, boundary 60/59, false, invalid). Contoh:

```
@Test
public void testCalculateGrade_A() {
    assertEquals("A", grade.calculateGrade(85));
}
@Test
public void testCalculateGrade_InvalidHigh() {
    assertThrows(IllegalArgumentException.class, () -> grade.calculateGrade(101));
}
@Test
public void testIsPassed_Boundary60() {
    assertTrue(grade.isPassed(60));
    assertFalse(grade.isPassed(59));
}
```

BAGIAN C: GITHUB ACTIONS (25 POIN)

Soal 5 (15 Poin) — Analisis workflow Python CI

1. Nama workflow

`Python CI` (baris `name: Python CI`).

2. Trigger/event

- `push` ke branch `main` dan `develop`.
- `pull_request` ke branch `main`.

Workflow berjalan setiap ada push ke 2 branch tersebut, dan setiap PR yang menargetkan `main`.

3. Fungsi `strategy.matrix`

Menjalankan **job yang sama secara paralel** untuk tiap kombinasi matrix — di sini 3 job paralel dengan `python-version: ['3.10', '3.11', '3.12']`. Tujuannya memastikan kode kompatibel di semua versi Python tanpa menduplikasi file YAML. Variabel `${{ matrix.python-version }}` dipakai di step setup Python.

4. Steps di job `test`

Dokumen soal menulis "5 steps", tetapi YAML yang ditampilkan hanya memuat **4 steps**:

13. `Checkout code` (`actions/checkout@v4`).
14. `Setup Python ${{ matrix.python-version }}` (`actions/setup-python@v5`).
15. `Install dependencies` (`pip install pytest pytest-cov`).
16. `Run tests` (`pytest --cov=src --cov-report=xml`).

(Tidak ada step ke-5 di YAML soal; pada materi kuliah, step ke-5 biasanya `Upload coverage report` dengan `actions/upload-artifact`.)

Soal 6 (10 Poin) — Workflow Java

File lengkap di repo: `.github/workflows/java-test.yml`

```
name: Java Unit Test

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Run tests
```

```
        run: mvn test

      - name: Generate coverage report
        run: mvn jacoco:report

      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco/
```

Sesuai ketentuan: nama `Java Unit Test`, trigger push main/develop + PR main, runner `ubuntu-latest`, dan 5 steps di atas (checkout, setup JDK 17, `mvn test`, `mvn jacoco:report`, upload artifact `jacoco-report`).

BAGIAN D: CODE COVERAGE & INTEGRASI (15 POIN)

Soal 7 — Analisis coverage

```
src/calculator.py    10   0   100%
src/login_system.py  15   2    87%
src/payment.py       20   8    60%
src/utils.py          5   1    80%
TOTAL                50  11    78%
```

1. Total statement: **50 (kolom `Stmts` baris TOTAL).**

2. Total miss (tidak tereksekusi): **11.**

3. Coverage paling rendah: **`src/payment.py`, 60%.**

4. Cara menaikkan coverage `payment.py` (8 miss dari 20):

- Jalankan `pytest --cov=src --cov-report=term-missing` untuk melihat baris yang miss.
- Tulis test tambahan untuk branch/path yang belum tersentuh (kasus gagal, exception, batas validasi, `else` yang belum dieksekusi) mengikuti pola AAA.
- Pastikan setiap `if/else`, loop, dan `raise` punya minimal satu test yang mengeksekusinya.
- Ulangi hingga file ≥ 80%.

5. Apakah 78% cukup baik?

Belum cukup untuk standar industri (target umum 80–90% line coverage).

Alasan: total 78% berarti 11 statement belum teruji; file kritis `payment.py` hanya 60% — modul pembayaran berisiko tinggi (uang/transaksi) sehingga bug yang lolos bisa fatal.

78% boleh diterima sebagai baseline sementara, tetapi harus dinaikkan ke ≥80% (ideal ≥85%) dengan prioritas ke `payment.py`, lalu `utils.py` dan `login_system.py`.

BAGIAN E: STUDI KASUS TERPADU — BONUS (10 POIN)

Implementasi lengkap di repo ini:

17. **Kode program:** `src/main/java/com/stmik/BankAccount.java`

- `deposit(double amount)`, `withdraw(double amount)`, `getBalance()`
- Validasi: amount harus > 0, withdraw tidak boleh melebihi saldo,

saldo awal tidak boleh negatif (melempar `IllegalArgumentException`).

18. **Unit test:** `src/test/java/com/stmik/BankAccountTest.java` — 8 test case

(saldo awal 0, deposit valid, deposit negatif/nol ditolak, withdraw valid,

withdraw melebihi saldo ditolak, withdraw negatif ditolak, konstruktor negatif ditolak).

19. **GitHub Actions:** `.github/workflows/java-test.yml` (menjalankan `mvn test`,

`mvn jacoco:report`, upload artifact `jacoco-report`).

20. **Repository:** https://github.com/Muhirabazram/latihan-git-1224014

- Pipeline: tab **Actions** → **Java Unit Test** harus hijau (sukses).
- Coverage: unduh artifact `jacoco-report`, buka `index.html`

(atau lihat `target/site/jacoco/` hasil `mvn jacoco:report` lokal).

Cara verifikasi lokal:

```
mvn test
mvn jacoco:report
```

LAMPIRAN: Perintah push repo ini (ringkas)

```
git add pom.xml src .github .gitignore Tugas
git commit -m "feat: add unit tests for pendaftaran, student grade, bank account"
git push origin main
```